

Comprehensive DSA Roadmap for Campus Internship & Placement Preparation

Week 1: Fundamentals & Arrays

Day 1: Understanding Time & Space Complexity

Focus Areas:

- Learn Big-O, Big-Ω, Big-Θ notations
- Understand how to analyze algorithms
- Practice analyzing simple loops and nested loops

Practice Problems:

1. *Easy:* [Time Complexity Analysis](#) - Analyze simple for loops
2. *Easy:* [What's the Time Complexity?](#) - Basic algorithm analysis
3. *Medium:* [Analyzing Recursive Algorithms](#) - Understanding recursion complexity
4. *Medium:* [Master Theorem Applications](#) - Solving recurrence relations
5. *Hard:* [Amortized Analysis](#) - Advanced complexity analysis

Day 2: Space Complexity & Algorithm Analysis

Focus Areas:

- Memory usage analysis
- Auxiliary space vs. input space
- Practical examples of space-efficient algorithms

Practice Problems:

1. *Easy:* [Space Complexity Quiz](#) - Basic space analysis
2. *Easy:* [Array Manipulation Space Analysis](#) - In-place operations
3. *Medium:* [Recursive Function Space](#) - Stack space in recursion
4. *Medium:* [Hash Map vs. Array](#) - Space tradeoffs
5. *Hard:* [Space Optimization Techniques](#) - Converting $O(n)$ to $O(1)$ space

Day 3: Introduction to Basic Algorithms

Focus Areas:

- Common algorithm patterns

- Comparing algorithm efficiencies
- When to use which approach

Practice Problems:

1. Easy: [Linear Search vs Binary Search](#) - Algorithm comparison
2. Easy: [Brute Force vs Optimized](#) - Different approaches
3. Medium: [Algorithm Selection Challenge](#) - Choose efficient algorithm
4. Medium: [Multiple Algorithm Solutions](#) - Compare approaches
5. Hard: [Efficiency Optimization](#) - Improving algorithms

Day 4: 1D Arrays Fundamentals

Focus Areas:

- Basic array operations
- In-place algorithms
- Array traversal techniques

Practice Problems:

1. Easy: [Array Reversal](#) - Basic manipulation
2. Easy: [Remove Element](#) - In-place operations
3. Medium: [Product Except Self](#) - Array manipulation
4. Medium: [Rotate Array](#) - In-place rotation
5. Hard: [First Missing Positive](#) - Advanced array manipulation

Day 5: 2D Arrays

Focus Areas:

- Matrix traversal patterns (row, column, diagonal)
- Matrix operations
- Simulating 2D arrays with 1D arrays

Practice Problems:

1. Easy: [Matrix Diagonal Sum](#) - Basic traversal
2. Easy: [Transpose Matrix](#) - Matrix operations
3. Medium: [Spiral Matrix](#) - Complex traversal
4. Medium: [Set Matrix Zeroes](#) - Matrix manipulation
5. Hard: [Word Search](#) - Advanced matrix traversal

Day 6: Sliding Window Technique

Focus Areas:

- Fixed-size window applications
- Variable-size window applications
- Window optimization techniques

Practice Problems:

1. *Easy:* [Max Average Subarray I](#) - Fixed window
2. *Easy:* [Contains Duplicate II](#) - Simple sliding window
3. *Medium:* [Longest Substring Without Repeating Characters](#) - Variable window
4. *Medium:* [Minimum Size Subarray Sum](#) - Sliding window optimization
5. *Hard:* [Minimum Window Substring](#) - Complex window problem

Day 7: Prefix Sum & Kadane's Algorithm

Focus Areas:

- Building prefix arrays
- Range sum queries
- Maximum subarray problems

Practice Problems:

1. *Easy:* [Range Sum Query - Immutable](#) - Basic prefix sum
2. *Easy:* [Maximum Subarray](#) - Kadane's Algorithm implementation
3. *Medium:* [Subarray Sum Equals K](#) - Prefix sum application
4. *Medium:* [Maximum Size Subarray Sum Equals k](#) - Advanced prefix sum
5. *Hard:* [Maximum Sum Circular Subarray](#) - Kadane's extension

Week 2: Arrays, Sorting & Searching

Day 8: Two Pointer Technique

Focus Areas:

- Two pointer patterns
- Applications for array problems
- Space optimization using pointers

Practice Problems:

1. *Easy:* [Two Sum Sorted Array](#) - Basic two pointers
2. *Easy:* [Remove Duplicates from Sorted Array](#) - In-place operation
3. *Medium:* [3Sum](#) - Multiple pointers

4. *Medium:* [Container With Most Water](#) - Two pointer optimization
5. *Hard:* [Trapping Rain Water](#) - Advanced two pointers

Day 9: Bubble & Selection Sort

Focus Areas:

- Bubble sort implementation and optimization
- Selection sort implementation
- Comparing elementary sorting algorithms

Practice Problems:

1. *Easy:* [Implement Bubble Sort](#) - Basic implementation
2. *Easy:* [Implement Selection Sort](#) - Basic implementation
3. *Medium:* [Sort Colors](#) - Dutch National Flag problem
4. *Medium:* [Relative Sort Array](#) - Custom sorting
5. *Hard:* [Sort an Array \(Implement from Scratch\)](#) - Implement multiple sorts

Day 10: Insertion & Merge Sort

Focus Areas:

- Insertion sort implementation and use cases
- Merge sort implementation and analysis
- Divide and conquer paradigm

Practice Problems:

1. *Easy:* [Implement Insertion Sort](#) - Basic implementation
2. *Easy:* [Implement Merge Sort](#) - Basic implementation
3. *Medium:* [Sort List](#) - Merge sort with linked lists
4. *Medium:* [Count Inversions](#) - Application of merge sort
5. *Hard:* [Count of Smaller Numbers After Self](#) - Advanced application

Day 11: Quick Sort & Counting Sort

Focus Areas:

- Quick sort implementation and optimization
- Partition strategies
- Counting sort for integer arrays

Practice Problems:

1. Easy: [Implement Quick Sort](#) - Basic implementation
2. Easy: [Sort Array By Parity](#) - Partition application
3. Medium: [Kth Largest Element in an Array](#) - Quickselect
4. Medium: [Top K Frequent Elements](#) - Counting + sorting
5. Hard: [Maximum Gap](#) - Linear time sorting

Day 12: Binary Search Fundamentals

Focus Areas:

- Binary search on sorted arrays
- Finding exact matches
- Template approach to binary search

Practice Problems:

1. Easy: [Binary Search](#) - Basic implementation
2. Easy: [First Bad Version](#) - Simple application
3. Medium: [Search in Rotated Sorted Array](#) - Modified binary search
4. Medium: [Find First and Last Position](#) - Range finding
5. Hard: [Median of Two Sorted Arrays](#) - Advanced binary search

Day 13: Binary Search Applications

Focus Areas:

- Binary search on answer
- Binary search on unsorted arrays
- Lower/upper bounds

Practice Problems:

1. Easy: [Guess Number Higher or Lower](#) - Interactive binary search
2. Easy: [Valid Perfect Square](#) - Binary search on answer
3. Medium: [Find Peak Element](#) - Binary search on unsorted array
4. Medium: [Capacity To Ship Packages Within D Days](#) - Binary search on answer
5. Hard: [Split Array Largest Sum](#) - Advanced binary search

Day 14: Advanced Search Applications

Focus Areas:

- Finding kth largest/smallest element
- Merge intervals
- Multi-dimensional search

Practice Problems:

1. *Easy:* [Intersection of Two Arrays](#) - Search application
2. *Easy:* [Valid Triangle Number](#) - Binary search approach
3. *Medium:* [Merge Intervals](#) - Interval handling
4. *Medium:* [Kth Smallest Element in a Sorted Matrix](#) - 2D search
5. *Hard:* [Swim in Rising Water](#) - Binary search on 2D grid

Week 3: Strings & Linked Lists

Day 15: String Basics

Focus Areas:

- String representation and operations
- Character manipulation
- String builder/buffer concepts

Practice Problems:

1. *Easy:* [Reverse String](#) - Basic operations
2. *Easy:* [Valid Palindrome](#) - Character checking
3. *Medium:* [Longest Palindromic Substring](#) - String manipulation
4. *Medium:* [String Compression](#) - In-place operations
5. *Hard:* [Distinct Subsequences](#) - Advanced string matching

Day 16: String Hashing & Matching

Focus Areas:

- Hash functions for strings
- Rabin-Karp algorithm
- String matching techniques

Practice Problems:

1. *Easy:* [Implement strStr\(\)](#) - String matching
2. *Easy:* [Jewels and Stones](#) - Hash set application
3. *Medium:* [Group Anagrams](#) - String hashing
4. *Medium:* [Repeated DNA Sequences](#) - Rolling hash
5. *Hard:* [Longest Duplicate Substring](#) - Advanced hashing

Day 17: Sliding Window for Strings

Focus Areas:

- Fixed and variable length windows
- Character frequency maps
- Optimization techniques

Practice Problems:

1. Easy: [Find All Anagrams in a String](#) - Fixed window
2. Easy: [Longest Common Prefix](#) - String comparison
3. Medium: [Longest Substring Without Repeating Characters](#) - Variable window
4. Medium: [Permutation in String](#) - Fixed window with frequencies
5. Hard: [Minimum Window Substring](#) - Complex window problem

Day 18: Two Pointers for Strings

Focus Areas:

- String traversal with pointers
- Palindrome problems
- Efficient string comparison

Practice Problems:

1. Easy: [Valid Palindrome II](#) - Almost palindrome
2. Easy: [Backspace String Compare](#) - Two pointer approach
3. Medium: [Palindromic Substrings](#) - Expanding around center
4. Medium: [3Sum](#) - Multi-pointer approach
5. Hard: [Interleaving String](#) - Two pointer DP approach

Day 19: Singly Linked Lists

Focus Areas:

- Linked list implementation
- Basic operations (insert, delete)
- Traversal patterns

Practice Problems:

1. Easy: [Reverse Linked List](#) - Basic operation
2. Easy: [Middle of the Linked List](#) - Fast/slow pointers
3. Medium: [Remove Nth Node From End](#) - Two-pass approach
4. Medium: [Linked List Cycle II](#) - Cycle detection
5. Hard: [Merge k Sorted Lists](#) - Divide and conquer

Day 20: Doubly & Circular Linked Lists

Focus Areas:

- Doubly linked list implementation
- Circular linked list operations
- Applications and advantages

Practice Problems:

1. *Easy:* [Design Linked List](#) - Implementation
2. *Easy:* [Palindrome Linked List](#) - Fast/slow approach
3. *Medium:* [LRU Cache](#) - Doubly linked list application
4. *Medium:* [Flatten a Multilevel Doubly Linked List](#) - Complex list operations
5. *Hard:* [All O`one Data Structure](#) - Doubly linked list design

Day 21: Advanced Linked List Operations

Focus Areas:

- Detecting and removing cycles
- Merging and sorting linked lists
- In-place reversals

Practice Problems:

1. *Easy:* [Remove Duplicates from Sorted List](#) - Basic operation
2. *Easy:* [Intersection of Two Linked Lists](#) - Finding common node
3. *Medium:* [Sort List](#) - Merge sort on linked list
4. *Medium:* [Rotate List](#) - Rotation operation
5. *Hard:* [Reverse Nodes in k-Group](#) - Complex reversal

Week 4: Recursion & Backtracking

Day 22: Recursion Basics

Focus Areas:

- Understanding recursion stack
- Base cases and recursive cases
- Tail recursion optimization

Practice Problems:

1. *Easy:* [Fibonacci Number](#) - Basic recursion
2. *Easy:* [Power of Three](#) - Simple recursive check
3. *Medium:* [Pow\(x, n\)](#) - Efficient recursive approach

4. *Medium:* [Unique Binary Search Trees II](#) - Recursive generation
5. *Hard:* [K-th Symbol in Grammar](#) - Complex recursion

Day 23: Divide and Conquer

Focus Areas:

- Divide and conquer paradigm
- Recursive problem decomposition
- Combining solutions

Practice Problems:

1. *Easy:* [Search in a Binary Search Tree](#) - Binary search via recursion
2. *Easy:* [Maximum Depth of Binary Tree](#) - Tree recursion
3. *Medium:* [Different Ways to Add Parentheses](#) - Recursive expression evaluation
4. *Medium:* [Sort an Array](#) - Implement merge sort
5. *Hard:* [Median of Two Sorted Arrays](#) - Divide and conquer approach

Day 24: Backtracking Introduction

Focus Areas:

- Backtracking paradigm
- State space tree exploration
- Decision making and undoing

Practice Problems:

1. *Easy:* [Letter Case Permutation](#) - Simple backtracking
2. *Easy:* [Binary Watch](#) - Combination problem
3. *Medium:* [Subsets](#) - Generate all subsets
4. *Medium:* [Permutations](#) - Generate all permutations
5. *Hard:* [N-Queens](#) - Classic backtracking problem

Day 25: Backtracking Applications

Focus Areas:

- Constraints and pruning
- Sudoku-like problems
- Combinatorial problems

Practice Problems:

1. Easy: [Sum of All Subset XOR Totals](#) - Subsets with operation
2. Easy: [Binary Tree Paths](#) - Path enumeration
3. Medium: [Combination Sum](#) - Sum combinations
4. Medium: [Word Search](#) - Grid search with backtracking
5. Hard: [Sudoku Solver](#) - Constraint satisfaction

Day 26: Classic Recursion Problems

Focus Areas:

- Tower of Hanoi
- Generating balanced parentheses
- Path finding problems

Practice Problems:

1. Easy: [Climbing Stairs](#) - Combinatorial counting
2. Easy: [Implement Tower of Hanoi](#) - Classic recursion
3. Medium: [Generate Parentheses](#) - Valid combinations
4. Medium: [Path Sum II](#) - Tree path finding
5. Hard: [Word Ladder II](#) - Path finding with backtracking

Day 27: Maze & Grid Problems

Focus Areas:

- Grid traversal algorithms
- Finding all paths
- Optimization in grid search

Practice Problems:

1. Easy: [Flood Fill](#) - Simple grid traversal
2. Easy: [Island Perimeter](#) - Grid counting
3. Medium: [Unique Paths](#) - Path counting
4. Medium: [Rat in a Maze](#) - Classic maze problem
5. Hard: [Word Search II](#) - Advanced grid search

Day 28: Advanced Backtracking

Focus Areas:

- Optimizing backtracking algorithms
- Pruning techniques
- Memoization with backtracking

Practice Problems:

1. Easy: [Path Sum](#) - Simple path check
2. Easy: [Find Mode in Binary Search Tree](#) - Tree traversal
3. Medium: [Palindrome Partitioning](#) - String partitioning
4. Medium: [Letter Combinations of a Phone Number](#) - Combination problem
5. Hard: [Remove Invalid Parentheses](#) - Optimized backtracking

Week 5: Stacks, Queues & Trees

Day 29: Stack Basics

Focus Areas:

- Stack implementation
- Push, pop, peek operations
- Stack applications

Practice Problems:

1. Easy: [Valid Parentheses](#) - Classic stack problem
2. Easy: [Implement Stack using Queue](#) - Data structure implementation
3. Medium: [Daily Temperatures](#) - Monotonic stack
4. Medium: [Evaluate Reverse Polish Notation](#) - Postfix evaluation
5. Hard: [Largest Rectangle in Histogram](#) - Complex stack application

Day 30: Stack Applications

Focus Areas:

- Balanced expressions
- Next/previous greater/smaller element
- Stack in algorithm design

Practice Problems:

1. Easy: [Backspace String Compare](#) - Stack simulation
2. Easy: [Remove All Adjacent Duplicates](#) - String stack
3. Medium: [Next Greater Element II](#) - Circular array stack
4. Medium: [Remove K Digits](#) - Monotonic stack
5. Hard: [Basic Calculator](#) - Expression evaluation

Day 31: Queue Basics

Focus Areas:

- Queue implementation
- Enqueue, dequeue operations
- Linear and circular queues

Practice Problems:

1. *Easy:* [Implement Queue using Stacks](#) - Data structure implementation
2. *Easy:* [Number of Recent Calls](#) - Simple queue application
3. *Medium:* [Design Circular Queue](#) - Circular implementation
4. *Medium:* [Perfect Squares](#) - BFS with queue
5. *Hard:* [Sliding Window Maximum](#) - Deque application

Day 32: Queue Applications & Deque

Focus Areas:

- Level order traversals
- BFS algorithm implementation
- Double-ended queue operations

Practice Problems:

1. *Easy:* [Moving Average from Data Stream](#) - Queue for sliding window
2. *Easy:* [Implement Stack using Queues](#) - Queue application
3. *Medium:* [Design Front Middle Back Queue](#) - Complex queue operations
4. *Medium:* [Shortest Path in Binary Matrix](#) - BFS with queue
5. *Hard:* [Sliding Window Maximum](#) - Deque optimization

Day 33: Binary Tree Basics

Focus Areas:

- Binary tree implementation
- Tree traversals (inorder, preorder, postorder)
- Binary tree properties

Practice Problems:

1. *Easy:* [Binary Tree Inorder Traversal](#) - Basic traversal
2. *Easy:* [Same Tree](#) - Tree comparison
3. *Medium:* [Binary Tree Zigzag Level Order Traversal](#) - Modified traversal
4. *Medium:* [Construct Binary Tree from Preorder and Inorder](#) - Tree construction
5. *Hard:* [Binary Tree Maximum Path Sum](#) - Tree DP

Day 34: Binary Search Trees

Focus Areas:

- BST properties and implementation
- Search, insert, delete operations
- BST validation

Practice Problems:

1. *Easy:* [Search in a Binary Search Tree](#) - Basic operation
2. *Easy:* [Minimum Absolute Difference in BST](#) - Inorder property
3. *Medium:* [Validate Binary Search Tree](#) - BST checking
4. *Medium:* [Kth Smallest Element in a BST](#) - Inorder traversal
5. *Hard:* [Recover Binary Search Tree](#) - BST correction

Day 35: Tree Traversal Techniques

Focus Areas:

- Recursive vs. iterative traversal
- Level order traversal
- Morris traversal

Practice Problems:

1. *Easy:* [Maximum Depth of Binary Tree](#) - Tree property
2. *Easy:* [Binary Tree Level Order Traversal](#) - BFS approach
3. *Medium:* [Binary Tree Preorder Traversal \(Iterative\)](#) - Stack-based traversal
4. *Medium:* [Lowest Common Ancestor of a Binary Tree](#) - Tree traversal application
5. *Hard:* [Binary Tree Cameras](#) - Advanced tree traversal

Week 6: Advanced Trees & Early Graph Concepts

Day 36: Tree Problems

Focus Areas:

- Diameter of binary tree
- Path sum problems
- Tree balancing

Practice Problems:

1. *Easy:* [Diameter of Binary Tree](#) - Tree measurement
2. *Easy:* [Symmetric Tree](#) - Tree structure checking
3. *Medium:* [Path Sum III](#) - Multiple paths

4. *Medium:* [Count Good Nodes in Binary Tree](#) - Path property checking
5. *Hard:* [Serialize and Deserialize Binary Tree](#) - Tree encoding

Day 37: Balanced BST & AVL Concepts

Focus Areas:

- Balanced tree properties
- Self-balancing techniques
- AVL tree rotations

Practice Problems:

1. *Easy:* [Balanced Binary Tree](#) - Height-balanced check
2. *Easy:* [Convert Sorted Array to Binary Search Tree](#) - Balanced construction
3. *Medium:* [Balance a Binary Search Tree](#) - Tree balancing
4. *Medium:* [Implement AVL Tree](#) - AVL implementation
5. *Hard:* [Count of Range Sum](#) - BST application

Day 38: Binary Tree Serialization

Focus Areas:

- Tree encoding/decoding
- Level order serialization
- Preorder serialization

Practice Problems:

1. *Easy:* [Construct String from Binary Tree](#) - Simple serialization
2. *Easy:* [Find Mode in Binary Search Tree](#) - Tree traversal
3. *Medium:* [Find Duplicate Subtrees](#) - Subtree serialization
4. *Medium:* [Construct Binary Tree from String](#) - String to tree
5. *Hard:* [Serialize and Deserialize Binary Tree](#) - Complete implementation

Day 39: Advanced Tree Traversals

Focus Areas:

- Zigzag traversal
- Boundary traversal
- Vertical order traversal

Practice Problems:

1. Easy: [Average of Levels in Binary Tree](#) - Level traversal
2. Easy: [Minimum Distance Between BST Nodes](#) - Inorder traversal
3. Medium: [Binary Tree Zigzag Level Order Traversal](#) - Modified BFS
4. Medium: [Vertical Order Traversal of a Binary Tree](#) - Complex traversal
5. Hard: [Binary Tree Cameras](#) - Postorder application

Day 40: Binary Heap

Focus Areas:

- Min/max heap implementation
- Heap operations
- Priority queue applications

Practice Problems:

1. Easy: [Last Stone Weight](#) - Heap application
2. Easy: [Kth Largest Element in a Stream](#) - Min heap
3. Medium: [Kth Largest Element in an Array](#) - Heap solution
4. Medium: [Ugly Number II](#) - Multi-source heap
5. Hard: [Find Median from Data Stream](#) - Two heaps

Day 41: Segment & Fenwick Trees

Focus Areas:

- Range query data structures
- Segment tree implementation
- Fenwick tree (Binary Indexed Tree)

Practice Problems:

1. Easy: [Range Sum Query - Immutable](#) - Prefix sum approach
2. Easy: [Implement Segment Tree](#) - Basic implementation
3. Medium: [Range Sum Query - Mutable](#) - Segment/Fenwick tree
4. Medium: [Count of Range Sum](#) - Segment tree application
5. Hard: [Count of Smaller Numbers After Self](#) - Fenwick tree

Day 42: Introduction to Graphs

Focus Areas:

- Graph representations (adjacency list, matrix)
- Graph properties
- Basic traversal concepts

Practice Problems:

1. Easy: [Find the Town Judge](#) - Degree counting
2. Easy: [Find Center of Star Graph](#) - Simple graph
3. Medium: [Graph Valid Tree](#) - Tree properties in graph
4. Medium: [Redundant Connection](#) - Cycle detection
5. Hard: [Critical Connections in a Network](#) - Bridge finding

Week 7: Graphs & Advanced Trees

Day 43: Graph Traversal: DFS

Focus Areas:

- Depth-first search implementation
- DFS applications
- Cycle detection

Practice Problems:

1. Easy: [Find if Path Exists in Graph](#) - Simple DFS
2. Easy: [Flood Fill](#) - Grid DFS
3. Medium: [All Paths From Source to Target](#) - Path finding
4. Medium: [Number of Islands](#) - Connected components
5. Hard: [Longest Increasing Path in a Matrix](#) - Matrix DFS

Day 44: Graph Traversal: BFS

Focus Areas:

- Breadth-first search implementation
- Level-order traversals
- Shortest paths in unweighted graphs

Practice Problems:

1. Easy: [Same Tree](#) - Tree BFS
2. Easy: [Minimum Depth of Binary Tree](#) - Tree BFS
3. Medium: [Shortest Path in Binary Matrix](#) - Grid BFS
4. Medium: [Rotting Oranges](#) - Multi-source BFS
5. Hard: [Word Ladder](#) - Graph construction and BFS

Day 45: Shortest Path Algorithms

Focus Areas:

- Dijkstra's algorithm
- Bellman-Ford algorithm
- Floyd-Warshall algorithm

Practice Problems:

1. Easy: [Find the City With the Smallest Number of Neighbors](#) - Floyd-Warshall
2. Easy: [Path With Minimum Effort](#) - Dijkstra
3. Medium: [Network Delay Time](#) - Dijkstra/Bellman-Ford
4. Medium: [Cheapest Flights Within K Stops](#) - Modified Bellman-Ford
5. Hard: [Path With Maximum Minimum Value](#) - Dijkstra variant

Day 46: Minimum Spanning Tree

Focus Areas:

- Prim's algorithm
- Kruskal's algorithm
- Union-Find data structure

Practice Problems:

1. Easy: [Connecting Cities With Minimum Cost](#) - MST application
2. Easy: [Implement Union-Find](#) - Disjoint Set implementation
3. Medium: [Min Cost to Connect All Points](#) - Kruskal's algorithm
4. Medium: [Redundant Connection](#) - Union-Find application
5. Hard: [Optimize Water Distribution in a Village](#) - Modified MST

Day 47: Advanced Graph Problems

Focus Areas:

- Topological sort
- Strongly connected components
- Bipartite graphs

Practice Problems:

1. Easy: [Is Graph Bipartite?](#) - Graph coloring
2. Easy: [Find Eventual Safe States](#) - Cycle detection
3. Medium: [Course Schedule](#) - Topological sort
4. Medium: [Reconstruct Itinerary](#) - Eulerian path
5. Hard: [Alien Dictionary](#) - Topological sort application

Day 48: Trie Basics

Focus Areas:

- Trie data structure implementation
- Insert, search, delete operations
- Prefix matching

Practice Problems:

1. *Easy:* [Implement Trie \(Prefix Tree\)](#) - Basic implementation
2. *Easy:* [Maximum XOR of Two Numbers in an Array](#) - Bit trie
3. *Medium:* [Design Add and Search Words Data Structure](#) - Trie with wildcards
4. *Medium:* [Replace Words](#) - Dictionary replacement
5. *Hard:* [Word Search II](#) - Trie with backtracking

Day 49: Trie Applications

Focus Areas:

- Autocomplete systems
- Spell checkers
- Word search optimizations

Practice Problems:

1. *Easy:* [Longest Word in Dictionary](#) - Trie traversal
2. *Easy:* [Map Sum Pairs](#) - Trie with values
3. *Medium:* [Word Search II](#) - Grid search with trie
4. *Medium:* [Design Search Autocomplete System](#) - Real-world application
5. *Hard:* [Stream of Characters](#) - Suffix trie

Week 8: Greedy, Dynamic Programming & Bit Manipulation

Day 50: Greedy Algorithm Basics

Focus Areas:

- Greedy approach principles
- When to use greedy algorithms
- Proving greedy correctness

Practice Problems:

1. *Easy:* [Assign Cookies](#) - Simple greedy
2. *Easy:* [Lemonade Change](#) - Coin change greedy

3. *Medium:* [Jump Game](#) - Reachability problem
4. *Medium:* [Gas Station](#) - Circular tour
5. *Hard:* [Candy](#) - Multi-pass greedy

Day 51: Greedy Applications

Focus Areas:

- Interval scheduling
- Huffman coding
- Activity selection

Practice Problems:

1. *Easy:* [Maximum Subarray](#) - Kadane's algorithm
2. *Easy:* [Partition Labels](#) - Interval partitioning
3. *Medium:* [Non-overlapping Intervals](#) - Interval scheduling
4. *Medium:* [Task Scheduler](#) - Process scheduling
5. *Hard:* [Minimum Number of Refueling Stops](#) - Greedy with heap

Day 52: Introduction to Dynamic Programming

Focus Areas:

- DP principles and paradigm
- Memoization vs. tabulation
- Building DP solution step by step

Practice Problems:

1. *Easy:* [Climbing Stairs](#) - 1D DP
2. *Easy:* [Min Cost Climbing Stairs](#) - 1D DP with cost
3. *Medium:* [House Robber](#) - Non-adjacent sum
4. *Medium:* [Coin Change](#) - Minimum coins
5. *Hard:* [Longest Valid Parentheses](#) - String DP

Day 53: 1D Dynamic Programming

Focus Areas:

- Linear DP problems
- State transitions
- Optimization problems

Practice Problems:

1. Easy: [Fibonacci Number](#) - Classic DP
2. Easy: [Divisor Game](#) - Game theory DP
3. Medium: [Longest Increasing Subsequence](#) - LIS problem
4. Medium: [Decode Ways](#) - String decoding
5. Hard: [Best Time to Buy and Sell Stock III](#) - State machine DP

Day 54: 2D Dynamic Programming

Focus Areas:

- Matrix DP problems
- Grid-based dynamic programming
- Path problems

Practice Problems:

1. Easy: [Unique Paths](#) - Grid counting
2. Easy: [Minimum Path Sum](#) - Grid path optimization
3. Medium: [Longest Common Subsequence](#) - String DP
4. Medium: [Edit Distance](#) - String modification
5. Hard: [Regular Expression Matching](#) - Pattern matching DP

Day 55: Advanced DP Problems

Focus Areas:

- Interval DP
- Knapsack problems
- State compression DP

Practice Problems:

1. Easy: [Maximum Subarray](#) - Kadane's algorithm
2. Easy: [Range Sum Query - Immutable](#) - Prefix sum
3. Medium: [Partition Equal Subset Sum](#) - 0/1 Knapsack
4. Medium: [Target Sum](#) - DP with signs
5. Hard: [Burst Balloons](#) - Interval DP

Day 56: Bit Manipulation Basics

Focus Areas:

- Bitwise operators (AND, OR, XOR, NOT)
- Bit shifting
- Common bit manipulation tricks

Practice Problems:

1. Easy: [Number of 1 Bits](#) - Bit counting
2. Easy: [Single Number](#) - XOR property
3. Medium: [Counting Bits](#) - Bit pattern recognition
4. Medium: [Reverse Bits](#) - Bit manipulation
5. Hard: [Minimum Number of Flips to Make a OR b Equal to c](#) - Bit comparison

Week 9: Advanced Topics & Interview Preparation

Day 57: Advanced Graph Algorithms

Focus Areas:

- Topological sorting
- Strongly connected components
- Network flow algorithms

Practice Problems:

1. Easy: [Find the Town Judge](#) - In/out degree
2. Easy: [Keys and Rooms](#) - Graph connectivity
3. Medium: [Course Schedule II](#) - Topological sort
4. Medium: [Evaluate Division](#) - Graph path product
5. Hard: [Reconstruct Itinerary](#) - Eulerian path

Day 58: Union-Find & Disjoint Set

Focus Areas:

- Union-Find implementation
- Path compression and rank optimization
- Applications in graph problems

Practice Problems:

1. Easy: [Number of Provinces](#) - Connected components
2. Easy: [Redundant Connection](#) - Cycle detection
3. Medium: [Most Stones Removed with Same Row or Column](#) - Connected islands
4. Medium: [Accounts Merge](#) - Graph clustering
5. Hard: [Swim in Rising Water](#) - Union-find with binary search

Day 59: Mock Interview Practice

Focus Areas:

- Problem-solving under time constraints
- Explaining thought process
- Handling edge cases

Practice Problems:

1. *Easy:* [Two Sum](#) - Array hash
2. *Easy:* [Valid Parentheses](#) - Stack problem
3. *Medium:* [LRU Cache](#) - Design problem
4. *Medium:* [Merge Intervals](#) - Sorting and merging
5. *Hard:* [Trapping Rain Water](#) - Two pointers/stack approach

Day 60: Final Review & Advanced Problem Solving

Focus Areas:

- Revisiting core algorithms
- Combining multiple techniques
- Time and space optimization

Practice Problems:

1. *Easy:* [Merge Two Sorted Lists](#) - Linked list merging
2. *Easy:* [Maximum Subarray](#) - Kadane's algorithm
3. *Medium:* [Word Break](#) - DP with strings
4. *Medium:* [Meeting Rooms II](#) - Interval scheduling
5. *Hard:* [Merge k Sorted Lists](#) - Multi-way merge

Additional Tips for Success:

1. *Consistent Practice:* Aim for at least 2-3 problems daily, focusing on quality over quantity
2. *Track Progress:* Keep a log of solved problems and revisit challenging ones
3. *Mock Interviews:* Use platforms like Pramp, InterviewBit, or ask friends to conduct mock interviews
4. *Time Management:* Practice with a timer to simulate interview conditions
5. *Implementation Focus:* Don't just understand solutions conceptually; implement them yourself
6. *Learn from Mistakes:* Analyze where you went wrong and what could be improved
7. *Revise Regularly:* Schedule weekly revision of core concepts and algorithms
8. *Join Communities:* Participate in coding communities to learn from others' experiences
9. *System Design:* Allocate some time to learn system design concepts as well

10. *Stay Healthy:* Maintain good sleep and exercise habits during your preparation